CS 610 Semester 2024–2025-I: Assignment 1

14th August 2024

Due Your assignment is due by Aug 26, 2024, 11:59 PM IST.

General Policies

- You should do this assignment ALONE.
- Do not copy or turn in solutions from other sources. You will be PENALIZED if caught.

Submission

- Submission will be through Canvas.
- Submit a compressed file called "⟨`roll`⟩`-assign1.tar.gz`". The compressed file should have the following structure.

```
-- roll
-- -- roll-assign1.pdf
-- -- <problem4-dir>
-- -- -- <source-files>
```

  The PDF file should contain solutions for the first three problems, and your description and results for the fourth problem. Show your computations where feasible and justify briefly.
- We encourage you to use the LaTeX typesetting system for generating the PDF file. You can use tools like Tikz, Inkscape, or Draw.io for drawing figures if required. You can alternatively upload a scanned copy of a handwritten solution, but MAKE SURE the submission is legible.
- You will get up to TWO LATE days to submit your assignment, with a 25% penalty for each day.

Problem 1 **[20 marks]**

Consider the following loop.

```
1    float s = 0.0, A[size];
2    int i, it, stride;
3    for (it = 0; it < 1000 * stride; it++) {
4        for (i = 0; i < size; i += stride) {
5            s += A[i];
6        }
7    }
```

Assume an 8-way set-associative cache with a capacity of 256 KB, line size of 64 B, and word size of 4 B (for `float`). The cache is empty before execution and uses an LRU replacement policy. Given `size`=32K, determine the total number of cache misses on `A` for the following access strides: 1, 4, 16, 64, 2K, 8K, 16K, and 32K. Consider all the three kinds of misses: cold, capacity, and conflict.

Solution

Size of cache `C` $= 2^{18}$ B
# of lines $= 2^{18}/64 = 2^{12}$
# of sets $= 2^{12}/8 = 2^9$
Size of `A` $= 32 * 2^{10} * 4 = 2^{17}$ B $<$ Size of cache `C`
There should not be any conflict misses because (i) the array `A` fits in the cache and (ii) 8-way associativity. So, the number of iterations by the outer loop has no impact.

Stride=1. $\frac{\texttt{size}}{\#\text{ words in a block}} = 32\text{K}/(64/4) = 2^{11}$

Stride=4. $\frac{\texttt{size}}{\#\text{ words in a block}} = 32\text{K}/(64/4) = 2^{112}$

Stride=16. $\frac{\texttt{size}}{\text{stride}} = 32\text{K}/16 = 2^{11}$

Stride=64. $\frac{\texttt{size}}{\text{stride}} = 32\text{K}/64 = 2^9$

Stride=2K. $\frac{\texttt{size}}{\text{stride}} = 32\text{K}/2\text{K} = 2^4$

Stride=8K. $\frac{\texttt{size}}{\text{stride}} = 32\text{K}/8\text{K} = 2^2$

Stride=16K. $\frac{\texttt{size}}{\text{stride}} = 32\text{K}/16\text{K} = 2$

Stride=32K. $\frac{\texttt{size}}{\text{stride}} = 32\text{K}/32\text{K} = 1$

Problem 2

[40 marks]

Consider a cache of size 64K words and lines of size 8 words. The matrix dimensions are 1024×1024. Perform cache miss analysis for the *kij* and the *jik* forms of matrix multiplication (shown below) considering direct-mapped and fully associative caches. The arrays are stored in row-major order. To simplify the analysis, ignore misses from cross-interference between elements of different arrays (i.e., perform the analysis for each array, ignoring accesses to the other arrays).

Listing 1: kij form

```
1  for (k = 0; k < N; k++)
2    for (i = 0; i < N; i++)
3      for (j = 0; j < N; j++)
4        C[i][j] += A[i][k] * B[k][j];
```

Listing 2: jik form

```
1  for (j = 0; j < N; j++)
2    for (i = 0; i < N; i++)
3      for (k = 0; k < N; k++)
4        C[i][j] += A[i][k] * B[k][j];
```

Your solution should have a table to summarize the total cache miss analysis for each loop nest variant and cache configuration, so there will be four tables in all. Justify your computations.

Loop	A	B	C
K	N	N	N
I	N	1	N
J	1	$\frac{N}{BL}$	$\frac{N}{BL}$
Total	N^2	$\frac{N^2}{BL}$	$\frac{N^3}{BL}$

(a) Direct-mapped cache

Loop	A	B	C
K	$\frac{N}{BL}$	N	N
I	N	1	N
J	1	$\frac{N}{BL}$	$\frac{N}{BL}$
Total	$\frac{N^2}{BL}$	$\frac{N^2}{BL}$	$\frac{N^3}{BL}$

(b) Fully-associative cache

Table 1: Cache miss analysis for the kij form.

Solution

BL is the number of elements that fit in a cache block.
Size of matrix $A/B/C = 2^{20}$ words
Size of cache = 2^{16} words
# of lines = $2^{16}/8 = 2^{13}$
# of sets in the direct-mapped cache = 2^{13}

Each array contains 2^{20} elements, while the cache can hold 2^{16} elements. Hence, only $\frac{1}{16}^{\text{th}}$ of an array can fit in the cache at a time. This means that elements (i, j) and $(i + 64, j)$ will map to the same cache block in a direct-mapped cache.

The address-to-cache-set mapping function is as follows. Given an address $A[i][j]$, the corresponding set number is given by

$$\text{Set \#} = \left(\lfloor \frac{(i * \#\text{cols} + j)}{BL} \rfloor \right) \bmod \#\text{sets}$$

Analysis of kij form for direct-mapped cache

Without loss of generality, let us assume that array element $A[0][0]$ maps to memory location 0. Since the block size is 8 words, $A[0][0] \ldots A[0][7]$ will occupy memory block 0, and map to the cache set 0. Elements $A[0][8] \ldots A[0][15]$ map to the cache set 1. With a consecutive set of 8 elements in a row occupying a memory block, the last element in row 0, $A[0][1023]$ would map to set (1024/8)-1=127. Wrapping around in row-major order, element $A[1][0]$ would map to set 128. Since the elements of row 1 would also occupy 128 blocks, $A[2][0]$ must map to the set $2 \times 128 = 256$. Generalizing, element $A[k][0]$ will map to set $[(k \times 128) \bmod 8K]$ since the number of sets in the direct-mapped cache is $8K$.

We will have a conflict and $A[0][1]$ will be evicted from the cache set 0 if any other element $A[k][0]$ also maps to set 0. The smallest value of k for this to happen is when $k \times 128$ equals 8×1024, i.e., $k = 64$. So when $A[64][0]$ is accessed, the new block loaded into set 0 will evict the previously-loaded block containing $A[0][0] \ldots A[0][7]$. Similarly, the block of data containing $A[65][0]$ will map to set 128 and cause eviction of the previously-loaded block containing $A[1][0] \ldots A[1][7]$.

Array `A`: It will have temporal reuse for the loop j; the only misses will be the initial cold misses for loop i for direct-mapped caches. All the misses will be repeated for the outer loop k because of conflicts. So, the total number of misses for the direct-mapped cache is N^2.

Array `B`: For a fixed k and i, there will be complete spatial reuse for loop j since it is accessed by a row in the innermost loop. There is temporal reuse for loop i for one row of the matrix. All

the misses will be repeated for the outer loop k. So, the number of misses for the direct-mapped cache is $\frac{N^2}{BL}$.

Array `C`: For a fixed k and i, there will be complete spatial reuse for loop j since it is accessed by a row in the innermost loop. As i is varied, the miss pattern will be repeated for each row. All the misses will be repeated for the outer loop k because of conflict misses with a direct-mapped cache. So, the total number of misses for the direct-mapped cache is $\frac{N^3}{BL}$.

Analysis of kij form for fully-associative cache

Array `A`: There is temporal reuse for the innermost loop j. As i is varied, the pattern of cold misses will be repeated for each row. There will not be conflict misses because N rows can fit in the fully-associative cache. There will be spatial locality for the outer loop k for each row. Hence, the total number of misses for the fully-associative cache is $\frac{N^2}{BL}$.

Array `B`: The total number of misses for the fully-associative cache is the same as in direct-mapped cache, $\frac{N^2}{BL}$.

Array `C`: The total number of misses for the fully-associative cache is the same as in direct-mapped cache, $\frac{N^3}{BL}$.

Analysis of jik form for direct-mapped cache

Array `A`: The array is accessed by row and so will have complete spatial reuse for loop k. The middle loop i causes different rows to be accessed. For a fixed j, the entire array is accessed once. As j is changed, the elements will have to be accessed again since the cache does not have sufficient capacity to enable any temporal reuse. So the number of misses is $\frac{N^3}{BL}$.

Array `B`: The array is accessed by column in the innermost loop k, resulting in N misses. All the misses are repeated for the i loop since the cache is of insufficient capacity and there are conflict misses. With a direct mapped cache, no spatial reuse is exploited for the outer loop j due to conflict misses in accessing the elements of each column. Hence, the miss cost is repeated for each distinct value of j, i.e., the multiplier is N. So the total number of misses for a direct-mapped cache will be N^3.

Array `C`: There will be complete temporal reuse since the inner loop index k does not appear in C's addressing. For a fixed j, as i is varied, a column of C is accessed. This will result in conflict misses for a direct-mapped cache. So the total number of misses for a direct-mapped cache will be N^2 (one miss for each element).

Analysis of jik form for fully-associative cache

Array `A`: The total number of misses for the fully-associative cache is the same as in direct-mapped cache, $\frac{N^3}{BL}$.

Array `B`: The array is accessed by column in the innermost loop k, resulting in N misses. For a fixed j, all the accesses are hits for the i loop for a fully-associative cache since the cache capacity is greater than N blocks. As j is changed from 0 to 1, no additional misses will occur as the ik loops are fully traversed. But we will encounter new data elements during the innermost loop's column access once every B iterations of j. Therefore, the multiplier for loop j is $\frac{N}{BL}$. So, the total number of misses for a fully-associative cache is $\frac{N^2}{BL}$.

Array `C`: There will be complete temporal reuse since the inner loop index k does not appear in C's addressing. For a fixed j, as i is varied, a column of C is accessed. This will not result in conflict misses for a fully-associative cache since the cache capacity is greater than N blocks. As

j is varied, we will encounter additional demand misses once for every BL iterations of j. So, the total number of misses for a fully-associative cache is $\frac{N^2}{BL}$.

Loop	A	B	C
J	N	N	N
I	N	N	N
K	$\frac{N}{BL}$	N	1
Total	$\frac{N^3}{BL}$	N^3	N^2

(a) Direct-mapped cache

Loop	A	B	C
J	N	$\frac{N}{BL}$	$\frac{N}{BL}$
I	N	1	N
K	$\frac{N}{BL}$	N	1
Total	$\frac{N^3}{BL}$	$\frac{N^2}{BL}$	$\frac{N^2}{BL}$

(b) Fully-associative cache

Table 2: Cache miss analysis for the jik form.

Problem 3

[30 marks]

Consider the following code.

```
#define N (2048)
double y[N], X[N][N], A[N][N];
for (k = 0; k < N; k++)
  for (j = 0; j < N; j++)
    for (i = 0; i < N; i++)
      y[i] = y[i] + A[i][j] * X[k][j];
```

Assume a direct-mapped cache of capacity 16 MB, with 32 B cache lines and a word of 8 B. Assume that there is negligible interference between the arrays `A`, `X`, and `y` (i.e., each array has its 16 MB cache for this question), and arrays are laid out in the row-major form.

Estimate the total number of cache misses for `A`, `X`, and `y`.

Solution

Size of 2D array X = 2^{25} bytes.
Size of cache = 2^{24} bytes.
Each cache block contains 32/8=4 words.
# of lines (or sets) = $16 * 2^{20}/32 = 2^{19}$

The arrays `X` and `A` are double the cache size, so we have to consider the possibility of capacity misses.
A[0][0] – A[0][3] $\implies$ Set 0
A[0][4] – A[0][7] $\implies$ Set 1
...
A[0][2044] – A[0][2047] $\implies$ Set 511
A[1][0] – A[1][3] $\implies$ Set 512
...
A[1023][2044] – A[1023][2047] $\implies$ Set $2^{19} - 1$

In other words, $k * 512 = 2^{19} \implies k = 2^{10}$. So there will be a conflict between lines `A`[0][0] and `A`[1024][0]. This can also be understood by the fact that only one-half of the array `A` or `X` can fit in the cache at a time ($\frac{16\text{MB}}{N \times N \times 8}$). So there will not be any reuse for the accesses to `A` or `X` because

of the j loop. There will be temporal reuse for the 2D array `X` for the inner loop i. The behavior with `X` is the same.

Every 4^{th} access will be a miss for the loop i for the vector `y`. Loops j and k will have contributions of 1 as the vector `y` fits in the cache.

Loop	**A**	**X**	**y**
K	N	N	1
J	N	$\frac{N}{B}$	1
I	N	1	$\frac{N}{B}$
Total	N^3	$\frac{N^2}{B}$	$\frac{N}{B}$

Table 3: Cache miss analysis for Problem 3.

\# of cache misses for `A` $= N^3 = (2^{11})^3 = 2^{33}$
\# of cache misses for `X` $= \frac{N^2}{B} = \frac{(2^{11})^2}{8} = 2^{19}$
\# of cache misses for `y` $= \frac{N}{B} = \frac{2^{11}}{8} = 2^8$

Problem 4 **[40 marks]**

Assume a naïve O(n^3) sequential matrix multiplication kernel (*ijk* form) for matrices of dimensions 2048×2048 (given).

(i) Implement an optimized version of the sequential matrix multiplication implementation using loop blocking. Experiment with different block dimensions (for e.g., 4×4 to 32×32).

Use a table to report the speedup of your optimized implementation over the sequential implementation for the different block sizes that you have tried. The first three columns will be the block sizes for matrices A, B, and C, and the last column will report the speedup compared to the sequential version.

(ii) Note the block size that works best for your setup. Remember that the cache hierarchy will influence the optimal block dimensions. Justify the improved performance by tracking performance counters with PAPI. List the PAPI version and the performance counters you have used.

- Do not change the `ijk` loop order.
- You may experiment with different block sizes for the three matrices.
- Create separate functions for your optimized variants for ease of debugging and evaluation.
- You should time your code on a noiseless system (i.e., no concurrent applications are running). Take times for 3–5 runs, and use the arithmetic mean.
- Report the cache hierarchy for the system you will use.
- Ensure that you are using relatively recent versions of PAPI (v5.7+).
- We will try to reproduce your results.